```
$ certbot certonly --server http://acme-lan.lan:8000/acme/directory -d db.example.net
```

acme-lan

Real, publicly-trusted certificates for the hosts the internet can never reach.

No relation to the anvil company. Although both are elaborate schemes that mostly work.

PyCon AU 2026 · Brisbane · lightning talk · github.com/smallsam/acme-lan · MIT

thisisunsafe

I have typed this so many times it stopped being a workaround and became a personality trait.

```
# the motivation, stated honestly
```

1 One name, two worlds

PUBLIC VIEW

`db.example.net`

Not reachable. No open ports. Possibly no A record at all.
The CA cannot knock on this door.

LAN VIEW

`db.example.net → 192.168.3.5`

Split-horizon DNS. A real public domain, pointed at a box that lives behind your firewall forever.

The CA does not need to reach the box. **It only needs proof you control the name.**

That proof is a TXT record. Which means someone has to hold DNS credentials.

```
# dns-01: prove the name, not the network path
```

2 The one prerequisite

Your internal names have to be public names.

NOT NEGOTIABLE

No .local. No .internal. No .lan.

A public CA will not certify a name nobody owns. You need a real, registered zone.

Good time to use that two-letter domain you have been hoarding.

GOOD NEWS

Split horizon stopped hurting

No second copy of the zone to keep in sync. Override only the names you host on the LAN; everything else resolves normally.

```
unbound   local-zone: "example.net." transparent
dnsmasq   address=/db.example.net/192.168.3.5
```

```
# one line per host, and the rest of your zone never notices
```

3 The three usual answers

OPTION A

DNS token on every host

Every box that needs a cert now holds a credential that can rewrite your whole zone.

Including the printer.

OPTION B

Run your own CA

Genuinely fine — until you have to install that root on every laptop, phone, CI runner and contractor.

Forever.

OPTION C

Self-signed and look away

The industry standard.

See the previous slide.

```
# all three work. none of them made me happy.
```

4 Why I built it anyway

0

appliances that will ever run certbot

1

machine holding your DNS credentials

90

days, renewed forever, without me

ESXi. iDRAC. The core switch. The NAS. The printer. None of them will ever pip install anything.

```
# the printer has never installed anything. the printer barely prints.
```

5 The whole idea

Be an ACME server.

Be an ACME client.

Sit in the middle.

CLIENT DIFF

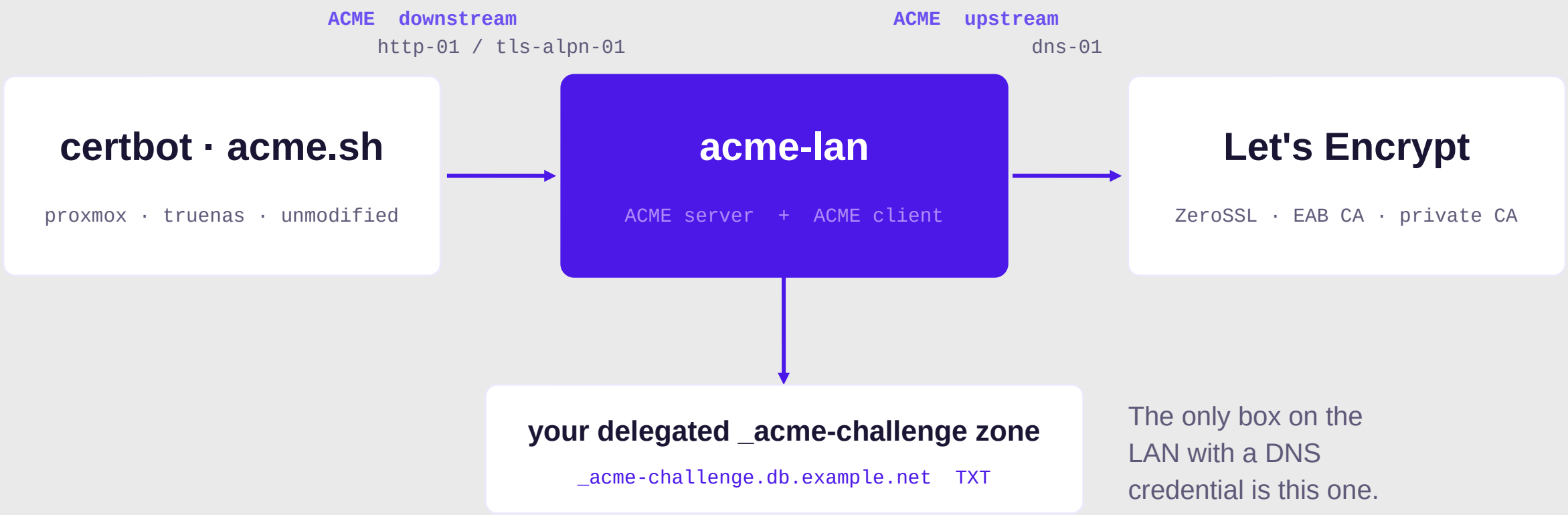
One line.

RFC 8555 both directions, so certbot, acme.sh and the ACME client already built into Proxmox need no plugin, no patch and no idea anything unusual is happening.

```
- --server https://acme-v02.api.letsencrypt.org/directory
+ --server http://acme-lan.lan:8000/acme/directory
```

```
# stock clients only. nothing that needs a custom hook script.
```

6 The architecture, all of it



The only box on the LAN with a DNS credential is this one.

```
# ~5k lines of Python standing between two ACME conversations
```

7 The one decision that matters

acme-lan forwards your CSR upstream.

- The issued cert matches the key the client already has
- acme-lan never sees, stores or transmits that private key
- Renewal is just ACME again — no state to keep in sync

```
order = upstream.new_order(csr_from_the_client)   # not one we made up
```

```
# a proxy that generated its own keys would be a very expensive self-signed cert
```

8 Two challenges, two directions

DOWNSTREAM · PROVE IT TO ME

http-01
tls-alpn-01

Port 80 if the box has a web server.

RFC 8737 over port 443 if it does not — the challenge rides in the TLS handshake itself.

Whichever listener the appliance already has.

UPSTREAM · PROVE IT TO THEM

dns-01
http-01 at the edge

dns-01 (default): nothing needs to be publicly reachable. Ever.

edge http-01: faster than DNS propagation, but you need a public IP and a wildcard A record.

One env var switches it.

```
ACME_LAN_UPSTREAM_CHALLENGE=dns-01
```

It’s always DNS.

The protocol designers agreed with you so completely they made a whole challenge type out of it.

But now one box holds a token that can rewrite your whole zone.

Better than thirty-seven copies of it. Still worth scoping down as far as your provider lets you.

9 Delegate the challenge, not the zone

```
_acme-challenge.db.example.net.    CNAME    db.acme-delegated.example.net.
```

WHAT ACME-LAN HOLDS

One narrow token

Write access to the delegated zone only. Nothing in the zone that runs your mail, your website or your MX records.

WHAT YOUR CLIENTS DO

Nothing different

The CA follows the CNAME on its own. It is in the spec, every provider supports it, and no client ever finds out.

One record per name, set once. That is the whole blast-radius story.

```
# works with cloudflare, route53, or whatever already runs your zone
```

10 Where acme-dns fits

SUPPORTED, AND GENUINELY NEAT

A DNS server with one opinion

Register once, add the CNAME, and acme-lan POSTs each TXT value to /update. Removal is a no-op — acme-dns expires them itself.

WHAT IT ADDS OVER A PLAIN CNAME

An internet-facing DNS server

One more service to run, patch and monitor — and on its own it needs clients that speak acme-dns, which most appliances never will.

Same blast radius as the CNAME you already made. More moving parts.

It is an alternative architecture, not an upgrade. Configure it if you already run acme-dns; otherwise delegate and move on.

```
ACME_LAN_DNS_PROVIDER=acmedns   # or cloudflare, if you would rather not
```

11 For the things that will never run ACME

MODE: DEVICE (PREFERRED)

The key never leaves the box

Fetch a CSR the device generated itself, sign it upstream, push back only the certificate.

Nothing secret ever crosses the wire.

MODE: LOCAL

We make the key and push both

For gear that cannot generate a CSR. The key is stored encrypted — never in plaintext at rest — and the UI tells you off about it anyway.

Install happens through deploy plugins: local (write files, run a reload) and ssh (SFTP + reload).
Most network gear is driven over SSH, so a vendor plugin is one class and a list of PluginFields.

ESXi · iDRAC · iLO · switches · printers · NAS · load balancers

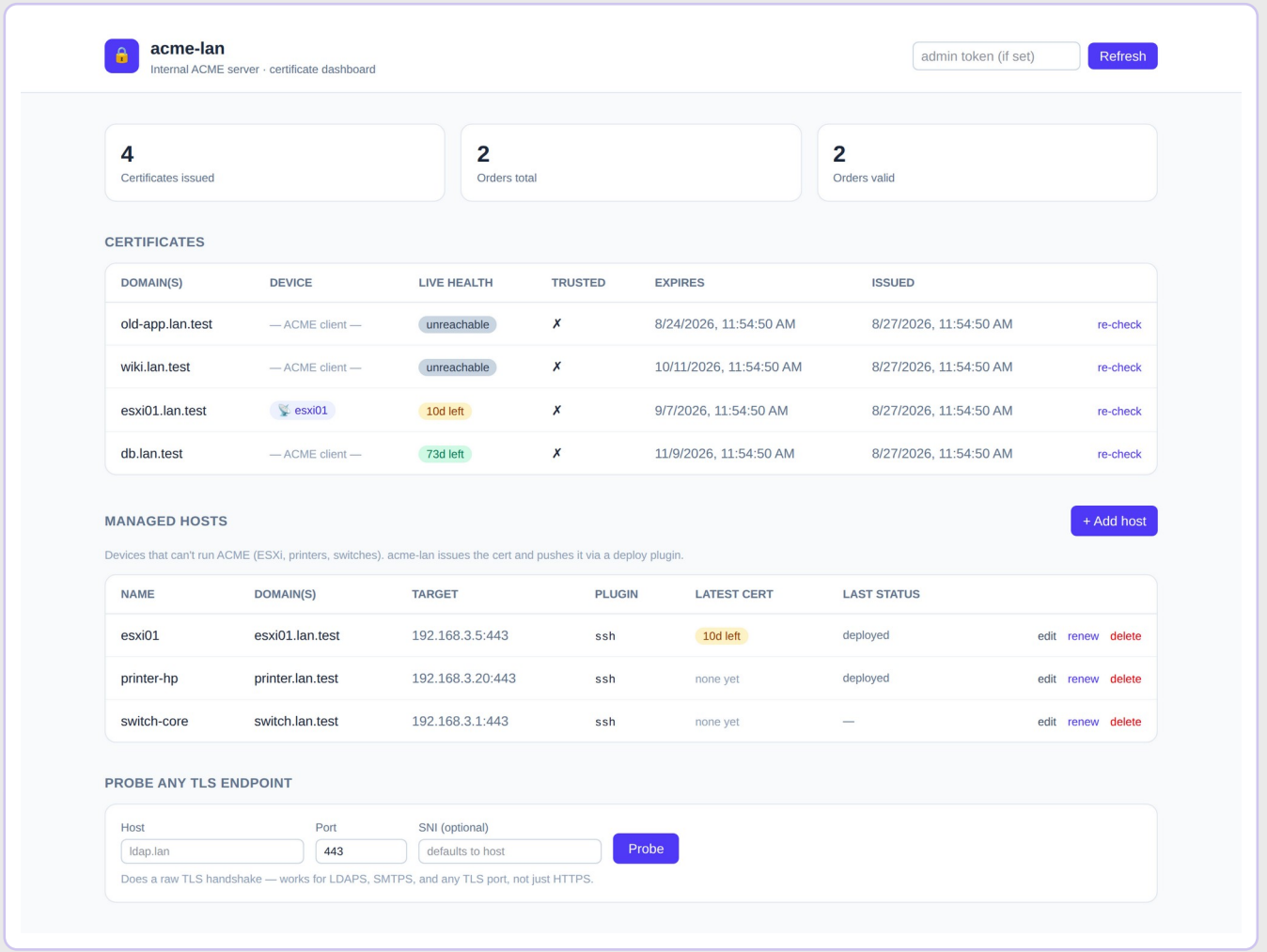
12 It comes with a dashboard

Every certificate, every managed host, one page.

Live health badges are a raw TLS probe, not a database lookup.

Certificates link back to the device they were pushed to, and back again.

Expiry warnings, retire-when-done, email + webhook notifications.

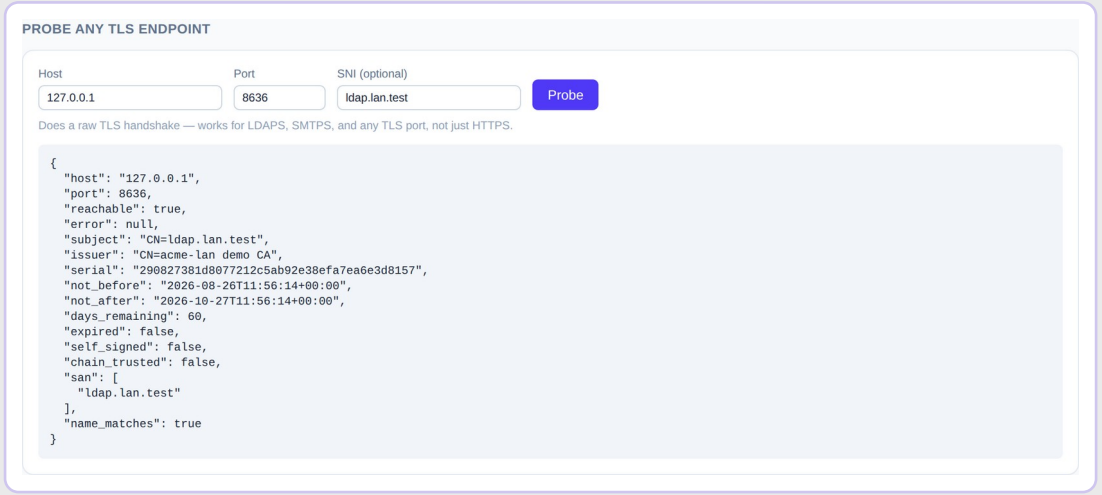



```
# GET /api/certificates · GET /api/hosts · POST /api/health/probe
```

13 Live health, not a spreadsheet

CERTIFICATES

DOMAIN(S)	DEVICE	LIVE HEALTH	TRUSTED	EXPIRES	ISSUED	
old-app.lan.test	— ACME client —	unreachable	✗	8/24/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
wiki.lan.test	— ACME client —	unreachable	✗	10/11/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
esxi01.lan.test	📡 esxi01	10d left	✗	9/7/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check
db.lan.test	— ACME client —	73d left	✗	11/9/2026, 11:54:50 AM	8/27/2026, 11:54:50 AM	re-check



A raw TLS handshake.

Not an HTTP request that assumes port 443 speaks HTTP.

LDAPS. SMTPS. That one appliance on 8443. Anything that completes a handshake gets an honest answer: expiry, chain trust, SAN match, self-signed.

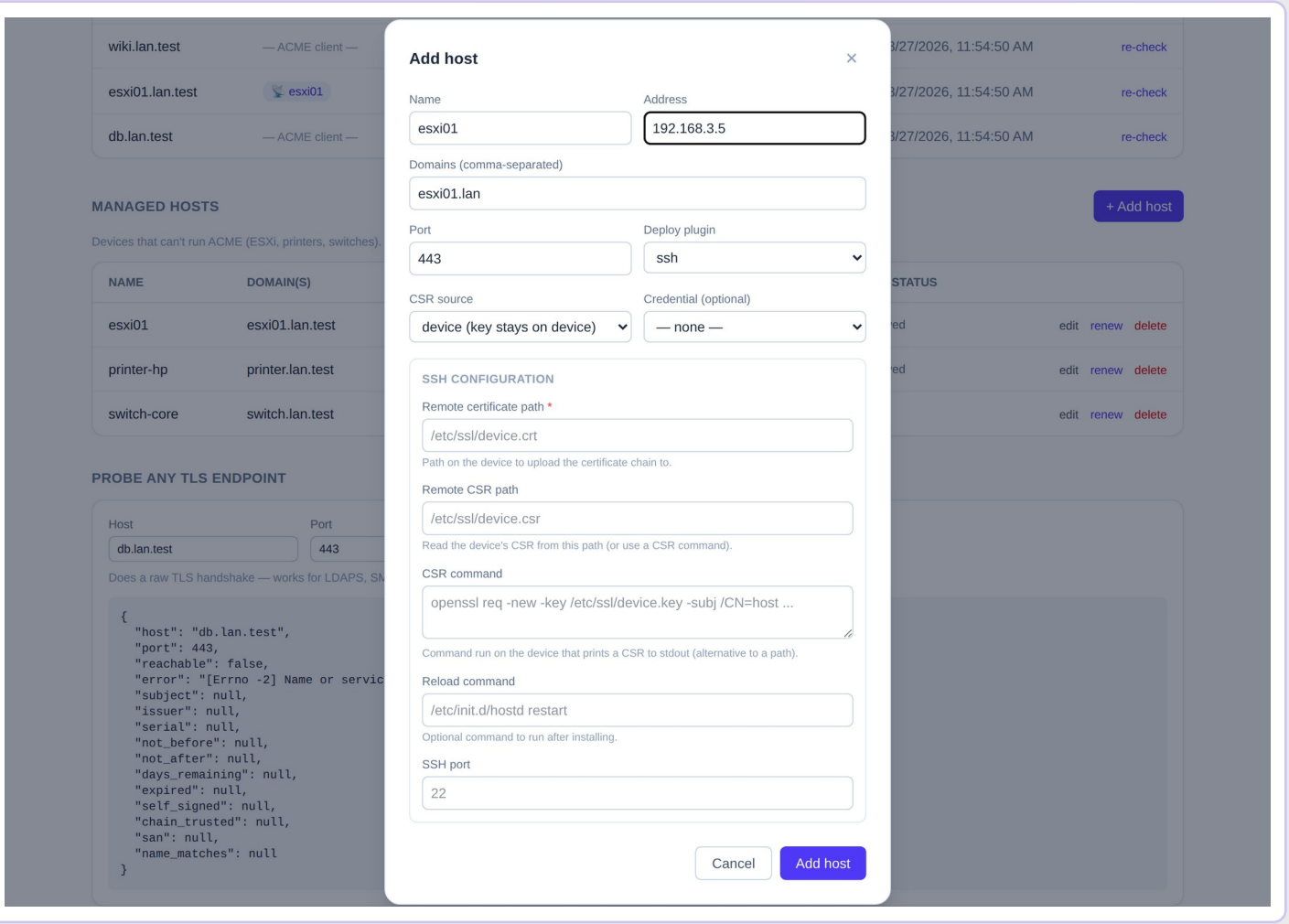
14 Adding a device shouldn’t be a JSON blob

Plugins declare their own config.

```
fields: ClassVar[list[PluginField]]
```

The dashboard reads them from the API and renders the real form — filtered by the plugin you picked and the CSR mode you chose.

Write a plugin, get a UI. No frontend involved.




```
# GET /api/deploy-plugins -> the form builds itself
```

15 Should you use this? Probably not.

Hosts reach the internet and can hold DNS creds	**certbot + a DNS plugin. Go home.**
Happy to run a private CA and push a root everywhere	**step-ca / smallstep. Genuinely excellent.**
Every client can speak acme-dns natively	**acme-dns on its own — if you like running authoritative DNS.**
One reverse proxy fronts everything you own	**Caddy or Traefik. Two lines of config.**
Appliances that can't run ACME, split-horizon names, one place to see it all	**…fine. That one is this.**

```
# a talk that tells you not to use the software is still a talk
```

16 The Python bits

FastAPI + async SQLAlchemy

One app. RFC 8555 endpoints, REST API, and the SPA, all from one process.

certbot's own acme library

The reference implementation is on PyPI. I did not write an ACME client.

cryptography

CSRs, signing, and the raw TLS probe behind every health badge.

Alembic on startup

The container migrates itself. Upgrades are docker pull.

Pebble + pytest

A real ACME CA in a container that issues deliberately terrible certs. e2e for the whole chain.

Vue 3 + Playwright

The dashboard, and UI tests in CI. Sorry — that bit is not Python.

```
# uv sync && uv run acme-lan
```

17 Three things I’d tell past me

1. **Forward the CSR.**
   Everything else in the design falls out of that one decision.

2. **Stay on staging longer.**
   Rate limits are a patient and thorough teacher.

3. **No plaintext keys at rest.**
   Fernet in the DB, or Key Vault, or Vault. Never a file on disk.

```
# the staging environment exists because I do
```

Try it

```
docker run -d --name acme-lan -p 8000:8000 -v acme-lan-data:/app/data \
  -e ACME_LAN_EXTERNAL_URL="http://acme-lan.example.net:8000" \
  -e ACME_LAN_SECRET_KEY="$(...fernet key...)" \
  -e ACME_LAN_DNS_PROVIDER="cloudflare" \
  ghcr.io/smallsam/acme-lan:latest      # defaults to LE staging
```

github.com/smallsam/acme-lan

MIT · docs for deployment, plugins and key handling in the repo · issues and vendor plugins very welcome

Thanks. No questions — lightning talk rules — but I'm in the hallway,
and my printer finally has a certificate nobody has to click through.